Università degli Studi di Trento
Corso di Programmazione I – a.a. 2014/2015
2° Appello - Prova scritta – 06/02/2015

Cognome.....
Nome.....
Numero Matricola.....

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
**NON** è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova..
Fa parte della valutazione la leggibilità del codice C/C++.
**Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su esse3.**
**Tempo a disposizione: 90 minuti.**

**[Domanda 1 - punti 3] Trovare gli errori logici e/o sintattici in ognuno dei seguenti segmenti di programma e spiegate come può' essere corretto:**

```
int sum ( int x, int y)
{
  int result;
  result = x+y;
}
```

```
void f( float a );
{
  float a;
  printf("%f",a);
}
```

```
int sum (int n) {
  if(n == 0) {
    return 0;
  } else{
    n+sum(n-1);
} }
```

**[Domanda 2 - punti 3] un tipo FILE è un tipo strutturato che contiene …**

**[Domanda 3]** Si consideri un programma che gestisce ricoveri ospedalieri.
Si vogliono immagazzinare informazioni relative ai pazienti che vengono ricoverati, quali nome, data di ricovero, data di dimissione, numero stanza, patologia, e nome del medico curante. Si assuma che i degenti siano organizzati in una lista semplicemente concatenata Una data è composta da giorno, mese e anno.

a) **[punti 3]** Definire opportune strutture dati per rappresentare le informazioni relative al paziente, alla lista ed alla data.
b) **[punti 2]** Definire una funzione **degenti** che, forniti come parametri di input in modo opportuno la lista dei pazienti, e una data, stampa i nomi di tutti i pazienti che risultano degenti (cioè ancora ricoverati) in tale data..
c) **[punti 3]** Definire una funzione **medico** che, forniti come parametri di input in modo opportuno la lista dei pazienti ed il nome di un medico, ritorna un array contenente le informazioni relative a tutti i pazienti curati da quel medico.
d) **[punti 2]** Definire una funzione **degenti_medico** che, dato in input in modo opportuno la lista di pazienti, una data, e il nome di un medico, stampa i nomi di tutti i pazienti di quel medico che (in nella data specificata) risultano degenti in ospedale. Si usino ove possibile le funzioni già definite.

Per i pazienti non ancora dimessi, si assuma la data di dimissione inizializzata al 1-1-2200.
Per effettuare il confronto tra stringhe si usi la funzione della libreria stringh.h
**int strcmp(const char* str1, const char *str2)** - restituisce 0 se str1 è uguale a str2.
Definire ulteriori funzioni e/o strutture che possono risultare utili e/o necessarie.
Si assuma che le funzioni operino su variabili, con tipo opportuno definito al punto a), riempite con i dati opportuni relativi ai pazienti.

**SOLUZIONE ESERCIZIO 3**

**[Domanda 4 - punti 6]**
**a) Applicare l'algoritmo di riempimento di un albero BST a questa sequenza di valori interi: disegnare l'albero così ottenuto, valori uguali DEVONO essere inseriti nel ramo di destra.**

| 3 | 1 | 3 | 8 | 0 | 5 | 6 |
|---|---|---|---|---|---|---|

**c) Per l'albero così realizzato inoltre calcolare**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Cammino | | | | | | | |
| Altezza | | | | | | | |
| Visita pre-ordine | | | | | | | |
| Visita post-ordine | | | | | | | |
| Visita in-ordine | | | | | | | |

**[Domanda 5 - punti 6]** Si scriva esattamente l'output (istruzioni ***printf***), nel riquadro a fianco, che viene prodotto durante l'esecuzione del programma se venisse invocata la funzione **MergeSort(A, 0, 4**), con A un array di interi.

```
void MergeSort (int A[], int p, int r){
        if (p < r) {
                q = floor((p+r)/2));
                printf("(p=%d, r=%d,q=%d)", p, r, q);
                MergeSort (A, p, q);
                MergeSort (A, q+1, r);
                Merge(A, p, q, r):
        }
}
```

**[Domanda 6 - punti 5]**

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 6
typedef int ArrayMatricola[MAX];
ArrayMatricola m;

int f1() {
  int a, b=3;
  for (a=0; a<MAX; a++)
   {
    m[a]/=2;
    if (m[a]<b) { b=m[a]; }
    printf("m[%d]=%d ",a,m[a]);
   }
  printf("\n b=%d \n", b);
  return b;
}

int f2(int* pos ) {
  *pos = (*pos) % MAX;
  printf("Pos=%d \n", *pos);
  printf("RESTO=%d\n", m[*pos]%2);
  if (m[*pos] % 2 == 0)
    { return m[0]+m[2]+m[4]; }
  else
    { return m[1]+m[3]+m[5]; }
 }

void main() {
  int i, j, k;
  //Inserisci le cifre della tua matricola, una alla volta
  for (i= MAX-1; i>=0; i--) scanf("%d", &m[i]);

  j=f1(); k=f2(&j);
  printf("j=%d k=%d\n",k,j);
  m[j]=k;
  for (i=MAX-1; i>=0; i--) { printf("%d ",m[i]); }
}
```

**DATI ESERCIZIO 6**
**MATRICOLA** ☐☐☐☐☐☐

Si scriva esattamente l'output (istruzioni ***printf***), nel riquadro a fianco, che viene prodotto durante l'esecuzione del programma.

**SOLUZIONE ESERCIZIO 6**

| |
|---|
| |
| |
| |
| |
| |
| |